

Lab 6. PowerShell as an Automation Tool

IT Infrastructure Management (W04IST–SI0826G)

Wojciech Thomas

Spring 2026

1 Lab Objectives

1. Become familiar with PowerShell as a scripting language

2 Introduction

To run scripts, it is best to use the PowerShell ISE environment. You can run the script you entered with the F5 key. You can execute the current line or selection with the F8 key.

Use Ctrl-I to switch to the editing window and Ctrl-D to switch to the execution window.

Remember these commands:

- `Get-Help <command> [-Examples | -Full | -Online]`
- `Get-Command <commandfragment>` – you can use wildcards `*`, for example `*net*`
- `Update-Help`

3 Exercises

3.1 Running a Script, Execution Policy

1. Log in to the **DC1** machine using the administrator account.
2. Start PowerShell ISE. Enter the code: `Write-Host "Hello!"`. Save the script in the default location under the name **hello.ps1**.
3. Open a PowerShell window as Administrator, go to the folder where you saved the script, and run it:

```
PS c:\Windows\System32> cd $env:USERPROFILE\Documents
PS c:\Users\Administrator\Documents > .\hello.ps1
```

4. Check the current script execution policy and change it to the default one:

```
Get-ExecutionPolicy
```

Set-ExecutionPolicy Default

Check what the default policy is.

5. Run the script again and review the displayed message.
6. Restore the ability to run local scripts:

Set-ExecutionPolicy RemoteSigned

7. Using autocomplete functionality (type `Set-ExecutionPolicy` and press TAB multiple times), check which script execution policy options are available.

3.2 Using Variables, Loops, and Conditional Statements

1. Add the command `param($Name)` at the beginning of the script.
2. Change the displayed text to `"Hello $($Name)!"`.
3. Run the script from the command line in the following ways:

```
./hello.ps1
./hello.ps1 Anna
./hello.ps1 -Name Anna
./hello.ps1 -<press TAB> ...
```

4. Using the construct `if(! $Name) { .... } else { ... }`, modify the script so that if no name is provided it displays `Hello stranger!`.
5. Using the construct `for($i = 1; $i -le 3; $i++) { ... }`, display the greeting several times.
6. Add the argument `$Count` to the parameter list, separated by a comma, to specify how many times the greeting should be displayed.
7. PowerShell is a language with strict type checking. Check what happens if you pass a word value as the `Count` parameter.
8. Modify the script parameter list so that the `Count` parameter can only be a number and defaults to 1: `[int] $Count=1`
9. Test the following script calls:

```
./hello.ps1 Anna
./hello.ps1 Anna -Count 6
./hello.ps1 Anna -Count six
```

10. Format the text using the `-f` option. You can find formatting examples at <https://ss64.com/ps/syntax-f-operator.html>.

3.3 Communication with the System

1. Run the command `Get-Process`.
2. Test result formatting with the commands `Format-List`, `Format-Wide`, and `Format-Table`, for example:

```
Get-Process | Format-List
Get-Process | Format-List cpu,name
```

The **Format-List** command has the alias **fl**, and **Format-Table** has the alias **ft**.

Remember that **Format-** commands transform a list of objects into text, so they should appear at the end of the pipeline.

3. Test result filtering using the **Where** and **Sort** filters, for example:

```
... | Where ProcessName -ge "notepad" | Sort ProcessName
```

In the **Where** command, try the operators **-lt** (that is <), **-le** (≤), **-gt** (>), **-ge** (≥), and **-like** **"*yst"**.

Find in the documentation the **Sort** option that determines the sort order.

4. Using the commands you have learned, display the processes consuming more than 1% of CPU time, sorted from the one placing the highest load on the CPU.
5. The **Where** command, which is an alias of **Where-Object**, also lets you define a filter as a script block in the following way (**\$_** means the current object):

```
... | Where { $_.ProcessName -ge "notepad" -and $_.CPU -le 0.5 }
```

6. Use the **ConvertTo-Html** and **ConvertTo-Csv** filters to transform the obtained results into HTML or CSV format respectively, and then use the **Out-File** **<filename>** command to save them to a file.
7. Start the Notepad application.
8. Using the instructions you have learned, find the process identifier of Notepad.
9. Execute the command:

```
$notes = (Get-Process | Where name -eq notepad)
```

What does the **\$notes** variable represent, that is, what object is it? Check which attributes and methods the **\$notes** object has: **\$notes | Get-Member**.

10. Find the method that closes the Notepad application. Call it as follows: **\$notes.<method>()**
11. Open several Notepad windows and repeat the experiment. Does the command close one window or multiple windows?
12. Log in to the **CL1** machine using the domain administrator account (remember that **DC1** and **CL1** must be running). Open PowerShell. Run **Get-Process** command using the **-ComputerName** option to display the list of processes running in the **DC1** machine. Check whether you can remotely run the command that closes the application.
13. Back in **DC1**, open a Notepad window. Execute the command

```
Stop-Process -Name notep* -Confirm
```

Compare the effect with the earlier steps.

14. Open several Notepad windows. Execute the command:

```
Stop-Process -Name notep* -WhatIf
```

What does the `-WhatIf` option do?

3.4 Bulk Account Creation

1. Create a file named `users.csv`:

```
FirstName,LastName,Department,Path
Anna,Test,IT,"OU=aaa,DC=bbb,DC=ccc"
```

2. Create the script `Create-Users.ps1`:

```
$users=Import-CSV users.csv
foreach ($i in $users) {
    Write-Host "Name: $($i.FirstName) $($i.LastName)"
}
```

3. Test the script.
4. Using the information obtained in previous labs, modify the script so that it creates accounts for all users listed in the file. The accounts should be enabled and have passwords set, which may be the same.